

The bionicreading package

Explorer

2026-04-27

Abstract

bionicreading highlights the leading part of western ASCII words. It provides a small $\text{\LaTeX}$ 3 interface backed by LuaTeX node processing. Both inline commands and paragraph environments mark their contents with a LuaTeX attribute; the Lua back-end later converts marked western glyphs during paragraph processing. CJK glyphs and mathematical material are not converted.

1 Overview

The package implements a bionic-reading-style visual treatment for western text. For each ASCII word, a calculated prefix is switched to the bold version of the current font. The calculation follows the fixation tables used by the text-vide project: higher fixation-point values generally produce shorter highlighted prefixes.

The package is intentionally scoped to western ASCII words. It does not attempt to segment CJK text, and it does not modify math lists. LuaLaTeX is required because all conversion is performed on the node list after macro expansion.

2 User interface

$\backslash$ bionicsetup	$\backslash$ bionicsetup{ <i>\langle options \rangle</i> }
--------------------------	--

Sets global package options. The currently supported keys are enabled and fixation-point.

$\backslash$ bionic	$\backslash$ bionic[<i>\langle options \rangle</i>]{ <i>\langle text \rangle</i> }
---------------------	--

Marks inline or short text for bionic-reading conversion. The command does not scan the argument as a TeX string. It sets a LuaTeX attribute while the argument is typeset; the Lua backend later processes the marked glyph nodes.

`\bionicstart` `\bionicstart[<options>]`

`\bionicstop` `<text>`

`\bionicstop`

Enables and disables marking for an explicit scope. The start command registers the current normal font and its bold counterpart as a font pair, then marks subsequently created glyphs with the current fixation point.

`bionicpar` `\begin{bionicpar}[<options>]`

`<paragraphs>`

`\end{bionicpar}`

A paragraph environment for bionic reading. The environment uses the same LuaTeX attribute mechanism as `\bionic`, but it also terminates the current paragraph at the end of the environment.

3 Options

Key	Value	Meaning
enabled	boolean	Enables or disables conversion.
fixation-point	1–5	Selects the fixation table.

4 Examples

```
\usepackage{bionicreading}
```

```
\bionic{Confucius said: Madam, I'm Adam.}
```

```
\bionic[fixation-point=5]{text-wide and internationalization}
```

```
\begin{bionicpar}[fixation-point=3]
```

```
\lipsum[1]
```

```
\end{bionicpar}
```

Both interfaces use the same LuaTeX node backend. Use `\bionic` for inline spans and `bionicpar` for paragraph blocks.

5 LuaTeX backend

The Lua backend scans paragraph node lists in `pre_linebreak_filter`. Only glyphs carrying the package attribute are converted. ASCII letters and digits are accumulated as one word. CJK glyphs, spaces, punctuation, math nodes, and other non-word nodes

terminate the current word. Kerning nodes and discretionary hyphenation nodes are not word boundaries: TeX can insert them inside western words, and treating them as boundaries would restart prefix highlighting in the middle of a word.

The backend changes only glyph font IDs. It does not insert new nodes and does not alter the textual content. This keeps macro expansion, CJK text, and math material under TeX’s normal control.

6 Implementation

6.1 Package file

```

1 <*package>
2 \NeedsTeXFormat{LaTeX2e}
3 \ProvidesExplPackage
4   {bionicreading}
5   {2026-04-27}
6   {0.1.0}
7   {Bionic-reading style highlighting for ASCII western text}
8
9 \RequirePackage{expl3}
10 \RequirePackage{xparse}
11
12 \ExplSyntaxOn
13
14 \msg_new:nnn { bionicreading } { invalid-fixation-point }
15   {
16     Invalid~fixation-point~'#1'.~
17     The~value-must-be-an-integer-from-1-to-5.
18   }
19 \msg_new:nnn { bionicreading } { luatex-required }
20   { The~bionicreading~package~requires~LuaLaTeX. }
21
22 \sys_if_engine luatex:F
23   { \msg_fatal:nn { bionicreading } { luatex-required } }
24
25 \RequirePackage{luatexbase}
26
27 \bool_new:N \l__bionicreading_enabled_bool
28 \int_new:N \l__bionicreading_fixation_point_int
29 \int_new:N \l__bionicreading_normal_font_int
30 \int_new:N \l__bionicreading_bold_font_int
31 \newluatexattribute \g__bionicreading_attribute

```

```

32
33 \keys_define:nn { bionicreading }
34 {
35     enabled .bool_set:N = \l__bionicreading_enabled_bool,
36     enabled .initial:n = true,
37     fixation-point .code:n =
38     {
39         \int_compare:nNnTF {#1} < { 1 }
40         { \msg_error:nnn { bionicreading } { invalid-fixation-point } {#1} }
41         {
42             \int_compare:nNnTF {#1} > { 5 }
43             { \msg_error:nnn { bionicreading } { invalid-fixation-point } {#1} }
44             { \int_set:Nn \l__bionicreading_fixation_point_int {#1} }
45         }
46     },
47     fixation-point .initial:n = 1,
48 }
49
50 \NewDocumentCommand \bionicsetup { m }
51 { \keys_set:nn { bionicreading } {#1} }
52
53 \cs_new_protected:Npn \__bionicreading_begin:n #1
54 {
55     \keys_set:nn { bionicreading } {#1}
56     \__bionicreading_lua_register_font_pair:
57     \__bionicreading_set_attribute:
58 }
59
60 \NewDocumentCommand \bionic { O{} +m }
61 {
62     \group_begin:
63     \__bionicreading_begin:n {#1}
64     #2
65     \group_end:
66 }
67
68 \directlua
69 {
70     bionicreading = dofile(kpse.find_file("bionicreading.lua"))
71     bionicreading.set_attribute(
72         luatexbase.attributes["g__bionicreading_attribute"]
73     )
74     bionicreading.register_callbacks()

```

```

75   }
76
77 \cs_new_protected:Npn \__bionicreading_set_attribute:
78 {
79   \bool_if:NTF \l__bionicreading_enabled_bool
80   {
81     \setluatexattribute \g__bionicreading_attribute
82     { \int_use:N \l__bionicreading_fixation_point_int }
83   }
84   { \unsetluatexattribute \g__bionicreading_attribute }
85 }
86
87 \cs_new_protected:Npn \__bionicreading_lua_register_font_pair:
88 {
89   \group_begin:
90     \int_set:Nn \l__bionicreading_normal_font_int { \fontid \font }
91     \bfseries
92     \int_set:Nn \l__bionicreading_bold_font_int { \fontid \font }
93     \directlua
94     {
95       bionicreading.register_font_pair(
96         \int_use:N \l__bionicreading_normal_font_int,
97         \int_use:N \l__bionicreading_bold_font_int
98       )
99     }
100   \group_end:
101 }
102
103 \NewDocumentCommand \bionicstart { O{} }
104 {
105   \group_begin:
106     \__bionicreading_begin:n {#1}
107 }
108
109 \NewDocumentCommand \bionicstop { }
110 { \group_end: }
111
112 \NewDocumentEnvironment { bionicpar } { O{} }
113 { \bionicstart [#1] }
114 { \par \bionicstop }
115
116 \ExplSyntaxOff
117 </package>

```

6.2 Lua backend

```
118  $\langle *lua \rangle$ 
119 local M = {}
120
121 local glyph_id = node.id("glyph")
122 local math_id = node.id("math")
123 local disc_id = node.id("disc")
124 local kern_id = node.id("kern")
125
126 local boundaries = {
127   { 0, 4, 12, 17, 24, 29, 35, 42, 48 },
128   { 1, 2, 7, 10, 13, 14, 19, 22, 25, 28, 31, 34, 37, 40, 43, 46, 49 },
129   { 1, 2, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47 },
130   { 0, 2, 4, 5, 6, 8, 9, 11, 14, 15, 17, 18, 20, 0, 21, 23, 24, 26, 27, 29, 30, 32, 33, 35, 37 },
131   { 0, 2, 3, 5, 6, 7, 8, 10, 11, 12, 14, 15, 17, 19, 20, 21, 23, 24, 25, 26, 28, 29, 30, 32, 34 },
132 }
133
134 M.attribute = nil
135 M.font_map = {}
136 M.callbacks_registered = false
137
138 local function is_ascii_letter(char)
139   return (char >= 65 and char <= 90) or (char >= 97 and char <= 122)
140 end
141
142 local function is_ascii_digit(char)
143   return char >= 48 and char <= 57
144 end
145
146 local function is_ascii_alnum(char)
147   return is_ascii_letter(char) or is_ascii_digit(char)
148 end
149
150 local function fixation_length(word_length, fixation_point)
151   local list = boundaries[fixation_point] or boundaries[1]
152   for index, boundary in ipairs(list) do
153     if word_length <= boundary then
154       local length = word_length - (index - 1)
155       if length < 0 then
156         return 0
157       end
158       return length
159     end
160   end
```

```

160     end
161     local length = word_length - #list
162     if length < 0 then
163         return 0
164     end
165     return length
166 end
167
168 local function node_marker(n)
169     if not M.attribute then
170         return nil
171     end
172     local marker = node.has_attribute(n, M.attribute)
173     if marker and marker >= 1 and marker <= 5 then
174         return marker
175     end
176     return nil
177 end
178
179 local function bold_word(word, has_letter, fixation_point)
180     if not has_letter or #word == 0 then
181         return
182     end
183     local bold_count = fixation_length(#word, fixation_point)
184     for i = 1, bold_count do
185         local glyph = word[i]
186         local bold_font = M.font_map[glyph.font]
187         if bold_font then
188             glyph.font = bold_font
189         end
190     end
191 end
192
193 function M.process_list(head)
194     if not M.attribute then
195         return head
196     end
197
198     local word = {}
199     local has_letter = false
200     local fixation_point = nil
201
202     local function flush()

```

```

203     if fixation_point then
204         bold_word(word, has_letter, fixation_point)
205     end
206     word = {}
207     has_letter = false
208     fixation_point = nil
209 end
210
211 for n in node.traverse(head) do
212     local marker = node_marker(n)
213     if n.id == glyph_id and marker and is_ascii_alnum(n.char) then
214         if fixation_point and fixation_point ~= marker then
215             flush()
216         end
217         fixation_point = marker
218         word[#word + 1] = n
219         if is_ascii_letter(n.char) then
220             has_letter = true
221         end
222     elseif n.id == kern_id or n.id == disc_id then
223         -- Font kerning and discretionary hyphenation can occur inside a word.
224         -- They must not restart fixation calculation for the following glyphs.
225     else
226         flush()
227     end
228 end
229
230 flush()
231 return head
232 end
233
234 function M.set_attribute(attribute)
235     M.attribute = tonumber(attribute)
236 end
237
238 function M.register_font_pair(normal_id, bold_id)
239     normal_id = tonumber(normal_id)
240     bold_id = tonumber(bold_id)
241     if normal_id and bold_id then
242         M.font_map[normal_id] = bold_id
243     end
244 end
245

```



```

246 function M.register_callbacks()
247   if M.callbacks_registered then
248     return
249   end
250   if luatexbase and luatexbase.add_to_callback then
251     luatexbase.add_to_callback("pre_linebreak_filter", M.process_list, "bionicreading")
252   else
253     callback.register("pre_linebreak_filter", M.process_list)
254   end
255   M.callbacks_registered = true
256 end
257
258 return M
259  $\langle$ /lua $\rangle$ 

```

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

B	
$\backslash$ bionic	<i>1, 2, 60</i>
bionicpar	<i>2</i>
bionicreading internal commands:	
$\backslash$ g_bionicreading_attribute	<i>31, 81, 84</i>
$\backslash$ _bionicreading_begin:n	<i>53, 63, 106</i>
$\backslash$ l_bionicreading_bold_font_int	
.....	<i>30, 92, 97</i>
$\backslash$ l_bionicreading_enabled_bool	
.....	<i>27, 35, 79</i>
$\backslash$ l_bionicreading_fixation_	
point_int	<i>28, 44, 82</i>
$\backslash$ _bionicreading_lua_register_	
font_pair:	<i>56, 87</i>
$\backslash$ l_bionicreading_normal_font_int	
.....	<i>29, 90, 96</i>
$\backslash$ _bionicreading_set_attribute:	
.....	<i>57, 77</i>
$\backslash$ bionicsetup	<i>1, 50</i>
$\backslash$ bionicstart	<i>2, 103, 113</i>
$\backslash$ bionicstop	<i>2, 109, 114</i>